Chương 3

HỌC TĂNG CƯỜNG VÀ TRÍ TUỆ NHÂN TẠO TẠO SINH

Mô hình máy học, không mất tính tổng quát, giả sử $f: \mathbb{R}^n \to \mathbb{R}$ là hàm thực n chiều, đã được giới thiệu trong chương trước có đặc điểm là với một véc-tơ đầu vào xác định $\boldsymbol{x}$, luôn chỉ cho một giá trị đầu ra $y = f(\boldsymbol{x})$, và như thế nếu nó đã sai với một mẫu nào đó thì nó vẫn cứ mãi sai nếu gặp lại chính mẫu đó. Hàm f như thế được gọi là phép biến đổi tất định (deterministic transformation). Một mô hình tất định không có khả năng tạo ra tri thức mới và sẽ được gọi là mô hình AI truyền thống để phân biệt với các mô hình có khả năng tạo ra cái mới, sẽ được gọi là trí tuệ nhân tạo tạo sinh (generative AI, Gen-AI), do có khả năng tăng cường tri thức mới vào tri thức đã được học trước đó. Tiếp cận học-tăng-cường là giải pháp để tăng cường kiến thức cho các hệ thống trí tuệ nhân tạo.

HỌC TĂNG CƯỜNG

Học tăng cường – REINFORCE Learning (RL), thực ra là cách chơi chữ của những người đã đề xuất phương pháp tăng cường tri thức cho hệ thống đã có. Từ REINFORCE được viết hoa vì đó là viết tắt của "đẳng thức"

Reward **I**ncrement = nonnegative **F**actor × **O**ffset **R**einforcement × **C**haracteristic **E**ligibility

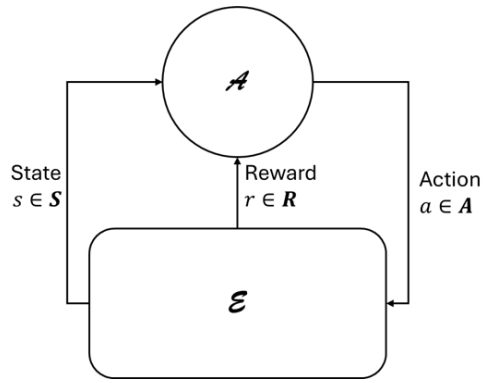
Định nghĩa hình thức về học tăng cường

Hệ sinh thái học tăng cường

Hệ sinh thái học tăng cường (REINFORCE Learning paradigm) duy trì một số tác nhân hoạt động trong một môi trường xác định trước. Không mất tính tổng quát, giả sử chỉ có một tác nhân $\mathcal{A}$ (agent) trong môi trường $\mathcal{E}$ (environment). Môi trường $\mathcal{E}$ được thể hiện, qua nhận thức của tác nhân $\mathcal{A}$, bằng một trạng thái trong tập trạng thái $\boldsymbol{S}$. Tác nhân tương tác với môi trường qua các hành động trong tập hành động $\boldsymbol{A}$ (action) được lập trình trước. Mỗi hành động của tác nhân sẽ được môi trường đáp ứng lại bằng một đáp ứng trong tập đáp ứng $\boldsymbol{R}$ (response), và có thể thay đổi trạng thái của môi trường.

Có thể hình dung $\mathcal{A}$ như một robot được trang bị các cảm ứng để có thể nhận biết trạng thái của môi trường, và một chương trình điều khiển robot chọn thực hiện hành động theo trạng thái nhận thức được. Trong ngữ cảnh robot chỉ có thể nhận thức được rõ ràng khi môi trường đáp ứng lại một hành động mà robot đã thực hiện. Vấn đề là làm sao để robot không "hối tiếc" (regret) về hành động nó đã thực hiện. Robot cần được trang bị một chương trình $\mathcal{L}$ có khả năng (tự) học một chiến lược (strategy) chọn hành động thích nghi tốt nhất với

tính ngẫu nhiên không đoán trước được của môi trường. Hình sau minh họa tác nhân $\mathcal{A}$ tương tác với môi trường sinh thái REINFORCE Learning paradigm.



Định nghĩa hình thức

Học tăng cường được định nghĩa là bộ 3 thuật toán ngẫu nhiên $RL = \langle S, A, R \rangle$, được định nghĩa trên trục thời gian $\boldsymbol{T}$, trong đó

. $S: \boldsymbol{T} \to \boldsymbol{S}$, với $\boldsymbol{S} = \{s_t = S(t)\} \neq \emptyset$ là không gian trạng thái. Hàm S nhận vào một giá trị thời điểm $t \in \boldsymbol{T}$, và thiết lập trạng thái của môi trường tại thời điểm đó, $s_i = S(t)$.

. $A: \boldsymbol{S} \to \boldsymbol{A}$, với $\boldsymbol{A} \neq \emptyset$ là tập các hành động được xác định trước. Hàm A là một hàm xác suất, nhận vào trạng thái s và trả về một hành động $a = A(s) \in \boldsymbol{A}$.

. $R: S \times A \times S \to \mathbb{R}$, nhận vào trạng thái $s \in \boldsymbol{S}$, xác định hành động $a \in \boldsymbol{A}$, trả về một giá trị $r \in \mathbb{R}$, đồng thời chuyển sang trạng thái mới $s' \in \boldsymbol{S}$. Giá trị $r > 0$ có thể xem như phần thưởng (reward) cho hành động tốt, trong khi, $r \leq 0$ biểu hiện sự "hối tiếc".

Học tăng cường như một trò chơi

Nếu như các phương pháp học khác, bao gồm học giám sát và học không giám sát, bài toán học được thiết kế như một bài toán tối ưu nhằm tìm kiếm một lời giải, thì học tăng cường lại thiết kế như một trò chơi nhằm tìm kiếm một cách chơi.

Học tăng cường với các học giám sát và không giám sát

Khác với học giám sát (supervisor learning) mà học trên dữ liệu được gán nhãn cho các nhiệm vụ định danh (identification), phân loại (classification), hay hồi quy dự báo (regression), … Học tăng cường dựa trên mục tiêu xác định rõ ràng trước, tác nhân nhận các tín hiệu do tương tác với môi trường để đạt đến mục tiêu cuối, và dùng cho các nhiệm vụ cần một chiến lược tối ưu (strategy) để đạt đến mục tiêu cuối.

Cũng khác với học không giám sát (unsupervison learning) mà tìm mối quan hệ không tường minh trong các dữ liệu không được gán nhãn trước cho các nhiệm vụ gom nhóm

(clustering), gãn nhãn hay đánh mã tự động (auto-labeling, auto-encoding), … Học tăng cường, với mục tiêu dài hạn xác định trước, tác nhân liên tục thăm dò để khám phá và tăng xác xuất đạt được mục tiêu dài hạn, và cho các nhiệm vụ mang tính tồn tại và phát triển.

Một cách trực quan, có thể hình dung tác nhân như một "sinh vật nhân tạo" (artificial organisms) "sống" trong môi trường sinh thái nhân tạo REINFORCE Learning. Có thể tóm tắt về REINFORCE Learning qua trò chơi tướng-cướp-một-tay.

Tướng cướp một tay

Tướng cướp một tay (one-armed bandit[1]), một truyện tranh nổi tiếng được viết lời bởi Bob de Groot và vẽ minh họa bởi Morris. Câu chuyện xoay quanh một máy đánh bạc với 1 tay gạt để người chơi có thể gạt chơi sau khi đã đặt xu vào máy.

Bối cảnh như sau: có M máy đánh bạc (slot machines), mỗi máy có xác suất phát thưởng nào đó. Người chơi có N xu mệnh giá 1 đô la, và mỗi lần chơi, có thể đặt 1 đồng xu vào 1 trong m máy đánh bạc. Mục tiêu là thu được càng nhiều tiền càng tốt sau N lần chơi. Tại mỗi lần chơi, nếu máy bỏ đồng xu được chọn, người chơi sẽ được thưởng tối đa K xu tương ứng rút ra từ phân phối xác suất của máy đó, nếu máy không được chọn, người chơi mất đồng xu đó. Trò chơi có thể được mô tả bằng hệ sinh thái REINFORCE Learning như sau.

Gọi

. $\mathcal{E}$ – môi trường gồm M máy đánh bạc (multi-armed bandits), mỗi máy có phân phối rút ra $k \in \{1, 2, \dots, K\}$ nào đó. Và máy thắng cuộc được chọn theo phân phối đều.

. $\mathcal{A}$ – robot có ngân sách N xu và có thể chơi n lần, mỗi lần đặt 1 xu vào 1 trong m máy.

REINFORCE Learning paradigm $RL = < S, A, R >$ được mô tả như sau:

. $S(t) = [m_1, \dots, m_M], \forall\, t = 1,2, \dots, N$, với $m_i = \begin{cases} 1, & rand(M) = i \\ 0, & otherwise \end{cases}$, trong đó $rand(X)$ là hàm ngẫu nhiên trả về 1 giá trị trong tập $\{1, 2, \dots, M\}$ với giả sử xác xuất đều nhau.

. $A(S(t)) = a_t$ là đặt đồng xu vào máy m_{a_t}.

[1] Bob de Groot (Author), Morris (Illustrator). The One-Armed Bandit (Lucky Luke). Cinebook, June 16, 2012.

. $R(S(t), A(S(t))) = \begin{cases} -1, & S(t)[A(S(t))] = 0 \\ Rand(K), & otherwise \end{cases}$, trong đó $Rand(Y)$ là hàm ngẫu nhiên trả về 1 giá trị trong tập $\{1,2, \dots, K\}$ với phân phối xác suất của máy chiến thắng.

Mục tiêu của A là "học" qua các lần chơi để có một chiến lược chơi sao cho chung cuộc thu được nhiều xu nhất sau N lần chơi:

$$\max_{A(S(t))} \Sigma_{t=1}^{N} R(S(t), A(t)).$$

Các chiến lược

Suy nghĩ đơn sơ nhất là cho tác nhân $\mathcal{A}$ thử chơi "đại" $n < N$ lần chơi để học biết quy luật, mà là phân phối của mỗi máy, sai đó, khai thác những khám phá này để chơi $N - n$ lần chơi còn lại. Toán bộ quá trình khám phá và khai thác đó được gọi là chiến lược (strategy) hay chính sách (policy) được cài đặt như thuật toán học cho $\mathcal{A}$.

Chiến lược tham lam (greedy strategy). Tại lần chơi thứ $t = 1,2, \dots$, máy được chọn là máy có trung bình phần thưởng đến thời điểm cao nhất với xác suất $1 - \varepsilon$. Trong trường hợp có nhiều máy cùng xác suất, chọn ngẫu nhiên 1 trong số các máy ứng viên này. Tham số $0 \leq \varepsilon \leq 1$, gần 1, $\mathcal{A}$ có khả năng khám phá tiếp, ngược lại, $\mathcal{A}$ đang chú trọng khai thác. Thuật giải cài đạt chiến lược này gọi là *Greedy Algorithm*.

Chiến lược khoảng tin cậy (confidence interval strategy). Thay vì chọn máy có phần thưởng trung bình cao nhất (với xác suất cao) như trong Greedy algorithm, gọi $Q^*(m_i) = E[R(S(t), A(S(t)))|A(S(t)) = m_i]$ là kỳ vọng phần thưởng máy m_i tính đến thời điểm t, chiến lược này gợi ý chọn máy có cận trên độ tin cậy (upper confidence bound – UCB) cao nhất. Thuật giải cài đặt chiến lược này gọi là UCB Algorithm. Khác với Greedy algorithm, ở đó tần suất khám phá và khai thác gần như không đổi do ε là hằng số, UCB algorithm cho phép tăng tần suất khai thác sau một khoảng thời gian thăm dò, nhưng vẫn duy trì khả năng khám phá của $\mathcal{A}$.

Chiến lược Bayes (Bayes-based strategy). Thay vì theo trường phái tần suất như 2 chiến lược trên, chiến lược dựa trên này dựa trên thống kê Bayes. Theo đó, suy luận để dự đoán về một giả thiết dựa trên dữ liệu mới dựa vào thông tin tiên nghiệm và dữ liệu quan sát được để đưa ra quyết định một cách linh hoạt, liên tục cập nhật dựa trên bằng chứng mới. Cụ thể chiến lược này sử dụng phân phối beta (beta distribution) để sinh ra một xác suất ngẫu nhiên, nhưng vẫn năm trong vùng cho phép, để dự đoán máy nào có thể sẽ cho phần thưởng trong lần chơi tiếp theo. Xác suất này được dựa trên kết quả của những lần chơi

trước đó. Chiến lược này sử dụng phương pháp lấy mẫu Thomson (Thomson sampling[2]) vên được gọi là Thomson Sampling. Phương pháp lấy mẫu Thomson sử dụng kỹ thuật thử-và-sai (try-and-error) để khám phá ra ra chiến lược chơi tốt nhất có thể. Ưu điểm của lấy mẫu Thomson là $\mathcal{A}$ có xu hướng khám phá theo hướng tìm kiếm (search-oriented) khi có ít dữ liệu, và giám khám phá khi có nhiều dữ liệu. Đây cũng là khác biệt chính giữa Thomson sampling so với UCB, ở đó hầu như chỉ có 1 lựa chọn dựa vào kết quả tổng hợp đến thời điểm đang xét, trong khi Thomson sampling tạo ra xác suất ngẫu nhiên trong khoảng cho phép nên có nhiều chọn lựa hơn.

Các kịch bản ứng dụng

Trò chơi tướng cướp một tay có thể được áp dụng khi có sự lựa chọn tuần tự giữa nhiều hành động và quan sát phần thưởng để đưa ra lựa chọn sáng suốt.

Đề xuất chọn nội dung. Có thể hình dung một trình web có thể chọn trong một số nội dung để đề xuất cho người dùng mỗi khi họ vào một trang web, và sẽ được thưởng bằng một cú nhấp chuột nếu người dùng hài lòng với đề xuất. Giả sử rằng nội dung trước đó không ảnh hưởng đến phản ứng của người dùng. Đây là một ứng dụng trực tiếp của tướng cướp một tay. Ví dụ này cũng có thể được mở rộng cho việc gửi email hoặc đề xuất sản phẩm.

Tìm email hay nội dung. Thay vì tối đa hóa phần thưởng tích lũy, mục tiêu bây giờ là tìm ra hành động tốt nhất: mô hình gọi chung là tướng cướp thám hiểm (exploration-bandit), hay mô hình này thử nghiệm A/B (A/B Testing). Chẳng hạn cần tìm ra phương pháp điều trị hiệu quả nhất trong số nhiều phương pháp điều trị một bệnh đã biết. Để thực hiện, chia trước quần thể thành hai nhóm: nhóm sẽ nhận phương pháp điều trị A và nhóm sẽ nhận phương pháp điều trị B. Khác với one-armed-bandit, exploration-bandit lần lượt chỉ định các cá thể cần điều trị cho phương pháp điều trị A hoặc B dựa trên phần thưởng đã quan sát được trên các cá thể được điều trị trước đó. Cách làm này cho phép tìm ra phương pháp điều trị tốt nhất nhanh hơn, hoặc đạt được kết quả có ý nghĩa hơn trong cùng một khoảng thời gian. A/B Testing này cũng phù hợp nếu bạn muốn thử nghiệm nhiều phiên bản email hoặc nội dung trước khi chọn phiên bản phù hợp để gửi đến đại đa số khách hàng. Tổng quát, có thể áp dụng để chọn các phương pháp xử lý phức tạp hơn.

Đấu giá thời gian thực. Việc lựa chọn giá thầu cho một đấu thầu thời gian thực cũng có thể được giải quyết bằng one-armed bandit. Các hành động sau đó là các giá thầu (có thể

[2] Thompson, W. R. (1935). On the theory of apportionment. American Journal of Mathematics, 57(2), 450-456.

là rời rạc hoặc liên tục, tùy thuộc vào lựa chọn mô hình) và phần thưởng là lượt nhấp chuột hoặc giao dịch mua.

Các thuật toán cho học tăng cường

Học tăng cường có mô hình và không có mô hình

Ở trên, để cho đơn giản, ta đã định nghĩa hệ sinh thái học tăng cường (REINFORCE Learning Paradigm) với môi trường $\mathcal{E}$, tác tử $\mathcal{A}$ có thể nhận biệt được môi trường qua trạng thái trong không gian trạng thái $\boldsymbol{S} = \{S: T \to \boldsymbol{S}\} \neq \emptyset$. Tuy nhiên, nhiều bài toán thực tế, việc biểu diễn được $\mathcal{E}$ là vấn đề phức tạp, thậm chí bất khả thi. Do đó, tùy thuộc tính chất của môi trường $\mathcal{E}$ mà có thể sử dụng thuật toán có môi trường hay không môi trường theo nghĩa có mô hình cho $\mathcal{E}$ hay không có mô hình xác định cho $\mathcal{E}$, gọi là học tăng cường dựa trên mô hình (Model-based RL) hoặc không mô hình (non-Model-based RL).

Model-based RL. Khi môi trường được xác định rõ ràng và không thay đổi, tác nhân $\mathcal{A}$ có thể xây dựng môi trường nội bộ để mô hình hóa môi trường $\mathcal{E}$ bằng cách thử thực hiện một số hành động khám phá môi trường, ghi nhận trạng thái môi trường và đáp ứng của môi trường qua phần thưởng nhận về. Sau đó, liên kết quá trình chuyển trạng thái với hành động bằng việc học phân phối chọn hành động, và sử dụng để khai thác môi trường.

Non-Model-based RL. Khi môi trường $\mathcal{E}$ không xác định và hay thay đổi, tác nhân $\mathcal{A}$, thay vì xây dựng môi trường nội bộ, $\mathcal{A}$ thực hiện các bước thử-sai (test-and-error) để quan sát các cặp hành động-trạng thái để phát triển chính sách "sinh tồn" trong $\mathcal{E}$.

Một số thuật toán cho học tăng cường phổ biến

REINFORCE

Ta bắt đầu với thuật toán REINFORCE, là tên thuật toán và cũng tên phương pháp được đề xuất bởi William[3]. Thuật toán REINFORCE là phương pháp lặp độ dốc chính sách (policy gradient) "củng cố - reinforce" các hành động dẫn đến kết quả tốt bằng cách tăng xác suất được lựa chọn sau này. Thuật toán REINFORCE đã đặt nền tảng cho nhiều phương pháp đạo hàm chính sách hiện đại và hiệu quả hơn, thường sử dụng các hàm giá trị đã học để giảm phương sai.

REINFORCE được áp dụng để học một chính sách ngẫu nhiên (randomized policy). Ký hiệu

[3] Ronald J. Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. Machine Learning, Volume 8, Issue 3-4, Pages 229 - 256, 01 May 1992 (1992)

. $\tau \equiv s_0 a_0 s_1 \dots s_{T-1} a_T s_T$, với $s_i \in \boldsymbol{S}, a_j \in \boldsymbol{A}$, và $t = 0,1,\dots,T < \infty$, là một quỹ đạo (trajectory) chiều dại hữu hạn T. Và

. $\pi_\theta(a|s)$ là xác suất chính sách π_θ chọn hành động a dưới điều kiện trạng thái s.

Giả sử có thể lấy mẫu từ các xác suất chuyển (transition probabilities) từ trạng thái s với hành động a, $\{P_{sa}\}$. Và nhận được phần thưởng $R(s,a)$. Mục tiêu của $\mathcal{A}$ là tại mỗi trạng thái hiện hành $s_t \in \boldsymbol{S}$, ở thời điểm t, chọn hành động $a_t \in \boldsymbol{A}$, sao cho kỳ vọng phần thưởng nhận được của quỹ đạo chiều dài T là cao nhất. Giả sử với trạng thái ban đầu s_0, hành động a_0 được chọn theo phân phối $\pi_\theta(a_0|s_0)$, hàm kỳ vọng phần thưởng được định nghĩa như

$\eta(\theta) = E[\Sigma_{t=0}^{T-1} \gamma^t R(s_t, a_t)]$, trong đó

$s_t \sim P_{s_{t-1} a_{t-1}}, a_t \sim \pi_\theta(.|s_t)$.

Bài toán học chính sách ngẫu nhiên được đưa về bài toán tối ưu

$\theta^* = \max_\theta \eta(\theta)$.

Khó khăn của ta ở đây là không biết hàm phần thưởng và xác suất chuyển. Đặt

. $P_\theta(\tau)$ là phân phối của τ, phát sinh bởi chính sách π_θ, và

. $f(\tau) = \Sigma_{t=0}^{T-1} \gamma^t R(s_t, a_t)$,

Hàm $\eta(\theta)$ có thể viết thành

$\eta(\theta) = E_{\tau \sim P_\theta}[f(\tau)]$.

Để ước lượng đạo hàm (gradient) của $\eta(\theta)$, tính đạo hàm $\eta(\theta)$ như sau:

$\nabla_\theta E_{\tau \sim P_\theta}[f(\tau)] = \nabla_\theta \int P_\theta(\tau) f(\tau) d\tau$

$= \int \nabla_\theta P_\theta(\tau) f(\tau) d\tau$

$= \int (\nabla_\theta P_\theta(\tau)) f(\tau) d\tau$

$= \int P_\theta(\tau)(\nabla_\theta \log P_\theta(\tau)) f(\tau) d\tau$ do $\nabla \log P_\theta(\tau) = \nabla P_\theta(\tau)/P_\theta(\tau)$.

$= \nabla_\theta E_{\tau \sim P_\theta}[(\nabla_\theta \log P_\theta(\tau)) f(\tau)]$.

Vậy ta có thể ước lượng đạo hàm $\nabla_\theta E_{\tau \sim P_\theta}[f(\tau)]$ qua $\nabla_\theta E_{\tau \sim P_\theta}[(\nabla_\theta \log P_\theta(\tau)) f(\tau)]$.

Gọi $\tau^{(i)}, i = 1, \dots, n$ là các mẫu quan sát được theo P_θ, có được từ việc thi hành chính sách π_0 n lần, mỗi lần T bước lặp. Đạo hàm của $\eta(\theta)$ được ước lượng theo

$\nabla_\theta E_{\tau\sim P_\theta}[f(\tau)] = \nabla_\theta E_{\tau\sim P_\theta}[(\nabla_\theta \log P_\theta(\tau))f(\tau)] \approx \frac{1}{n}\Sigma_{i=1}^{n}\left(\nabla_\theta \log P_\theta\left(\tau^{(i)}\right)\right)f\left(\tau^{(i)}\right)$.

Trong công thức xấp xỉ trên, ta cần tính $\log P_\theta(\tau)$ và đạo hàm của nó theo θ.

Từ định nghĩa của τ, ta có

$P_\theta(\tau) = \mu(s_0)\pi_\theta(a_0|s_0)P_{s_0a_0}(s_1)\pi_\theta(a_1|s_1)P_{s_1a_1}(s_2) \dots P_{s_{T-1}a_{T-1}}(s_T)$,

trong đó μ là hàm phân phối mật độ của s_0. Do đó, lấy log 2 vế phương trình trên,

$\log P_\theta(\tau) = \log \mu(s_0) + \log \pi_\theta(a_0|s_0) + \log P_{s_0a_0}(s_1) + \cdots + \log P_{s_{T-1}a_{T-1}}(s_T)$.

Và ta có thể tính đạo hàm của $\log P_\theta(\tau)$,

$\nabla_\theta \log P_\theta(\tau) = \nabla_\theta \log \pi_\theta(a_0|s_0) + \nabla_\theta \log \pi_\theta(a_1|s_1) + \cdots + \nabla_\theta \log \pi_\theta(a_{T-1}|s_{T-1})$.

Cuối cùng, ta có thể ước lượng đạo hàm kỳ vọng từ các mẫu trên quỹ đạo:

$\nabla_\theta \eta(\theta) = \nabla_\theta E_{\tau\sim P_\theta}[f(\tau)]$

$= E_{\tau\sim P_\theta}\Sigma_{t=0}^{T-1}(\nabla_\theta \log \pi_\theta(a_t|s_t))f(\tau)$

$= E_{\tau\sim P_\theta}[(\Sigma_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t|s_t)) \cdot (\Sigma_{t=0}^{T-1}\gamma^t R(s_t, a_t))]$.

Thuật toán đạo hàm chính sách được Vanilla thiết kế như sau, trong đó $R_\geq$ ký hiệu cho $\Sigma_{j\geq t}^{T-1}\gamma^t R(s_j, a_j)$.

Algorithm (Vanilla algorithm):

$For\ i = 1,2, \dots$:

. Thu thập tập các quỹ đạo bằng cách thi hành chính sách hiện hành.

. Tìm hàm B cực tiểu hóa tổng $\Sigma_\tau\Sigma_t\left(R_{\geq t} - B(s_t)\right)^2$.

. Cập nhật tham số θ với ước lượng đạo hàm $\Sigma_\tau\Sigma_t(\nabla_\theta \log \pi_\theta(a_t|s_t)) \cdot (R_{\geq t} - B(s_t))$.

Thuật toán Q

Thuật toán Q (Q-algorithm) thường dùng để chỉ Q-learning, một thuật toán học tăng cường không mô hình (non-Model RL), huấn luyện tác nhân $\mathcal{A}$ tìm ra chính sách cho phép chọn hành động tối ưu bằng cách học kỳ chất lượng, thường gọi là giá trị Q (Q-value) khi thực hiện một hành động ứng với một trạng thái môi trường $\mathcal{E}$. Thuật toán Q là thuật toán lặp, duy trì bảng Q-table và liên tục cập nhật mỗi cặp các giá trị giá trị-trạng thái, dựa vào phần

thưởng hiện hành nhận được và phần thưởng kỳ vọng lớn nhất, cho đến khi hội tụ về một chính sách tối ưu. Công thức cập nhật sử dụng phương trình Bellman[4]:

$$Q(s,a) = Q(s,a) + \alpha[\gamma \max_{a'} Q(s',a') - Q(s,a)],$$

trong đó, α là hệ số học quyết định mức độ thông tin mới ảnh hưởng đến giá trị Q cũ như thế nào; và γ là hệ số chiết khấu cân bằng giữa phần thưởng trước mắt và phần thưởng trong tương lai.

Xét bài toán tối ưu

$$a^* = \max_a Q(t,a) + c\sqrt{\frac{\ln(t)}{N_t(a)}},$$

trong đó

. $N_t(a)$ là số lần hành động a được chọn cho đến thời điểm t.

. $Q(t,a) = \frac{\textit{tổng số phần thưởng nhận được khi a được chọn}}{\textit{tổng số lần a được chọn}}$.

Algorithm (Q-learning):

Step 1: Định nghĩa môi trường $\boldsymbol{\mathcal{E}}$

Môi trường $\boldsymbol{\mathcal{E}}$ = <$\boldsymbol{S}, \boldsymbol{A}, a_{goal}, Q_{table}$>, trong đó

. $\boldsymbol{S}$ là tập các trạng thái,

. $\boldsymbol{A}$ là tập các hành động,

. $a_{goal} \in \boldsymbol{A}$ là trạng thái đích cần đạt, và

. Q_{table} là bảng giá trị Q.

Step 2: Xác định các tham số

. γ: hệ số chiết khấu,

. α: hệ số học, và

. ε: xác suất chọn hành động.

Step 3: Định nghĩa hàm chuyển trạng thái

[4] Richard Ernest Bellman. Dynamic Programming. Courier Corporation, 2003.

Hàm chuyển trạng thái tính toán trạng thái kế dựa vào trạng thái hiện hành và hành động được chọn.

Step 4: Lặp cập nhật bảng Q

Lặp:

. Tính $a^* = \max_a Q(t,a) + c\sqrt{\frac{\ln(t)}{N_t(a)}}$

. Cập nhật $Q(s,a) = Q(s,a) + \alpha[\gamma \max_{a'} Q(s',a') - Q(s,a)]$.

Mạng học sâu Q

Mạng học sâu Q (Deep Q-Network) là một mở rộng của Q-learning khi môi trường $\mathcal{E}$ có không gian trạng thái lớn khiến duy trì bảng Q là bất khả thi. Trong những trường hợp như vậy, bảng Q được thay bằng một mang nơ-ron để ước lượng giá trị Q.

SARSA-learning

SARSA, viết tắt của State-Action-Reward-State-Action là thuật toán huân luyên tác nhân $\mathcal{A}$ theo chính sách thay vì khám phá môi trường $\mathcal{E}$ như Q-learning. Phương pháp này cập nhật trực tiếp chính sách để tối đa hóa phần thưởng. REINFORCE ta đã học trên là thuật toán SARSA.

HỌC TĂNG CƯỜNG TRONG TRÍ TUỆ NHÂN TẠO TẠO SINH

Mô hình trí tuệ nhân tạo tạo sinh

Các phương pháp ML-AI được giới thiệu trên có thể học ra các mô hình tất định (deterministic model), nghĩa là nếu f là mô hình học được thì với một đầu vào $\boldsymbol{x}$ cố định, luôn chỉ cho ra một đầu ra $y = f(\boldsymbol{x})$. Như vậy, f chỉ có thể tái hiện "gần đúng" những gì đã được học. Nghĩa là, nếu $\varepsilon > 0$ là sai số của mô hình[5], thì nếu $\boldsymbol{x}$ là một trong những mẫu đã được sử dụng trọng quá trình huấn luyện mô hình f, bao gồm cả dữ liệu học (training dataset) $\Omega_{training}$ lẫn dữ liệu đánh giá (validation dataset) $\Omega_{validation}$, $f(\boldsymbol{x}) \leq \varepsilon$; ngược lại, nếu $\boldsymbol{x} \notin \Omega = \Omega_{training} \cup \Omega_{validation}$, thì $f(\boldsymbol{x})$ được xem là kết quả tính toán xấp xỉ của đầu vào $\boldsymbol{x}$, hay là dự đoán (prediction) cho x. Ta sẽ gọi các mô hình ML-AI này là trí tuệ nhân tạo dự đoán (Predictive-AI hay Pred-AI) để phân biệt với các mô hình trí tuệ nhân tạo tạo sinh (Generative-AI hay Gen-AI) mà ta sẽ học trong mục này.

---

[5] Sai số của mô hình là tham số được dùng để đánh giá hiệu suất (performance/accuracy) của mô hình qua các dữ liệu đánh giá (validation dataset). Tham số này cũng thường được sử dụng để dừng quá trình huấn luyện mô hình.

Một mô hình trí tuệ nhân tạo tạo sinh (Gen-AI) là mô hình có khả năng tạo ra các nội dung mới từ dữ liệu đầu vào dựa trên những gì nó đã được học trước. Khác với Pred-AI mà mục tiêu chính là phân tích dữ liệu (cho nhận dạng, phân loại, hay dự đoán nói chung), Gen-AI có thể được xem là một tác nhân $\mathcal{A}$ (agent) được trang bị các thuật toán máy học để khái phá (mining) ra các mẫu (pattern) trong dữ liệu học (dataset), rồi sử dụng "kiến thức" (knowledge) đó để "sáng tạo", thực ra là tổng hợp tạo ra (generate) ra các nội dung mới.

Từ Pred-AI đến Gen-AI

Ngẫu nhiên hóa hàm tất định

Như ta đã thấy ở trên, Pred-AI là mô hình tất định, trong khi Gen-AI thì không. Mô hình Gen-AI f cần một phép biến đổi ngẫu nhiên (stochastic transformation). Giải pháp đơn giản để tất định thành ngẫu nhiên là "đính kèm" với $\boldsymbol{x}$ một biến ngẫu nhiên $\boldsymbol{z}$ lấy mẫu theo một phân phối nào đó, như phân phối đều (uniform distribution) hay phân phối chuẩn (Gauss distribution) chẳng hạn. Như vậy, phần mạng nơ-ron vẫn thực hiện các phép tính tất định nhưng hàm $f(\boldsymbol{x}, \boldsymbol{z})$ sẽ ngẫu nhiên với dữ liệu quan trắc không có quyền truy cập vào $\boldsymbol{z}$. Nếu f liên tục và khả vi, có thể sử dụng các phương pháp huấn luyện, thường là phương pháp lan truyền-ngược (back-propagation), cho học xây dựng mô hình. Chẳng hạn, xét toán tử lấy mẫu theo phân phối chuẩn

$$y = N(\mu, \sigma^2).$$

Vì mẫu y không phải được tạo ra từ một hàm tất định, nhưng từ tiến trình lấy mẫu với xác xuất thay đổi. Không thể lấy đạo hàm theo μ và σ^2, tuy vậy, có thể viết lại tiến trình lấy mẫu như phép biến đổi biến ngẫu nhiên $z \sim N(z; 0,1)$ để có mẫu

$$y = \mu + \sigma z.$$

Bây giờ ta có thể huấn luyện bằng cách coi mẫu $y = \mu + \sigma z$ như phép tính tất định (deterministic operation) với biến gắn thêm z vào.

Lưu ý quan trọng là đầu vào $\boldsymbol{x}$ có gắn thêm z sẽ là một biến ngẫu nhiên mà phân phối của $\boldsymbol{x}||z$, ($\boldsymbol{x}$ gắn z), không phải là một hàm của bất kỳ biến nào mà ta cần tính đạo hàm. Một thay đổi dù nhỏ của μ hoặc σ cũng sẽ thay đổi đầu ra nếu lặp lại thao tác lấy mẫu với cùng giá trị z.

Học mô hình không tất định

Ở trên, ta đã biết giải pháp ngẫu nhiên hóa một hàm tất định bằng cách thêm yếu tố ngẫu nhiên và dữ liệu để có thể sử dụng lại các công cụ toán học cũng như tin học để huấn luyện một mô hình AI không tất định (non-deterministic model).

Trong huấn luyện mô hình tất định, không mất tính tổng quát, có thể xem mục tiêu của huấn luyện là cực tiểu hóa sai số $\mathcal{L}(\theta)$ của mô hình tham số hóa, bằng cách giải bài toán tối ưu hàm

$$\theta^* = \min_{\theta} \mathcal{L}(\theta),$$

bằng kỹ thuật giảm đạo hàm.

Trong kỹ thuật giảm đạo hàm, việc tính được giá trị đạo hàm $\nabla\mathcal{L}(\theta)$ là nhiệm vụ cốt lõi của phương pháp. Tuy nhiên, trong ngữ cảnh ngẫu nhiên (stochastic) của Gen-AI, cần một số thao tác kỹ thuật để có thể tính được đạo hàm hàm lỗi.

Khi đầu ra của mô hình Gen-AI, f_θ, là y với xác suất $p(y|\boldsymbol{x};\theta)$. Ký hiệu

$$\boldsymbol{\omega} \cong \boldsymbol{\omega}(\theta, \boldsymbol{x})$$

là biến ngẫu nhiên. Biến này là sự kết hợp giữa tham số mô hình θ và đầu vào $\boldsymbol{x}$.

Cho y được lấy mẫu từ phân phối $p(\mu|\boldsymbol{\omega})$, với $\boldsymbol{\omega} = \boldsymbol{\omega}(\theta, \boldsymbol{x})$, thì phân phối $y \sim p(y|\boldsymbol{\omega})$ có thể viết lại thành

$$y = f(z; \boldsymbol{\omega}),$$

với z là một nguồn ngẫu nhiên.

Bằng cách thiết kế hàm phần thưởng phù hợp, bài toán tạo sinh sẽ là bài toán huấn luyên-kép (dual training) gồm

(1) Cực tiểu hóa sai số như học trong mô hình Pred-AI: $\theta^* = \min_{\theta} \mathcal{L}(\theta)$, và

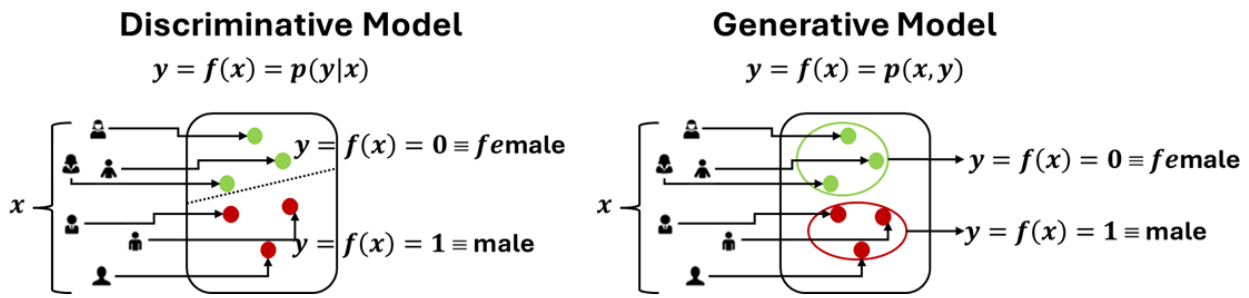
(2) Cực đại hóa kỳ vọng phần thưởng đạt được của RL: $E[\Sigma_t \gamma^t R(s_t, a_t)]$.

Bài toán (2) có thể giải bằng các thuật toán của học tăng cường như ta đã học ở trên.

HAI MÔ HÌNH TẠO SINH CƠ BẢN

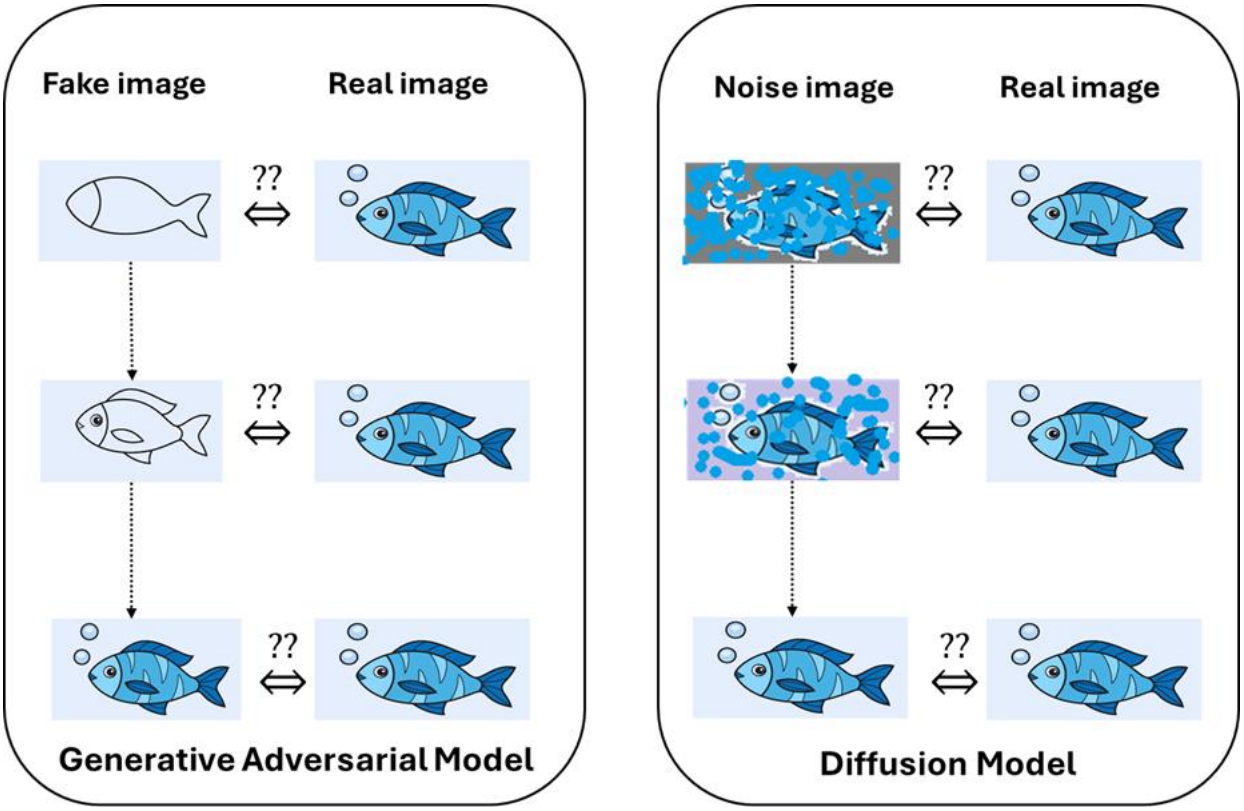
Phương pháp tạo sinh cơ bản

Không minh tính tổng quát, để minh họa cách phân biệt giữa mô hình AI tất định (deterministic model) và mô hình AI không tất định (non-deterministic model), ta giới hạn mô hình tất định là mô hình với nhiệm vụ nhận dạng hay phân biệt một đối tượng đầu vào x, gọi là mô hình phân biệt (discriminative model); và mô hình không tất định là mô hình tạo sinh (generative model) như hình vẽ.



Về mặt hình thức, discriminative model tính giá trị xác suất đối tượng đầu vào x thuộc lớp y, cho mọi lớp hệ thống được học, mà là xác suất có điều kiện $f(x) = p(y|x)$. Trong khi đó, generative model thì lại tính giá trị của biến ngẫu nhiên $\omega = x||y$ được ghép bởi x và y, $f(x) = p(x, y)$.

Để dễ trình bày, ta xem xét 2 tiếp cận cơ bản cho các mô hình tạo sinh: mô hình đối kháng tạo sinh (generative adversarial model) và mô hình phát tán (diffusion model) như minh họa trong hình vẽ.

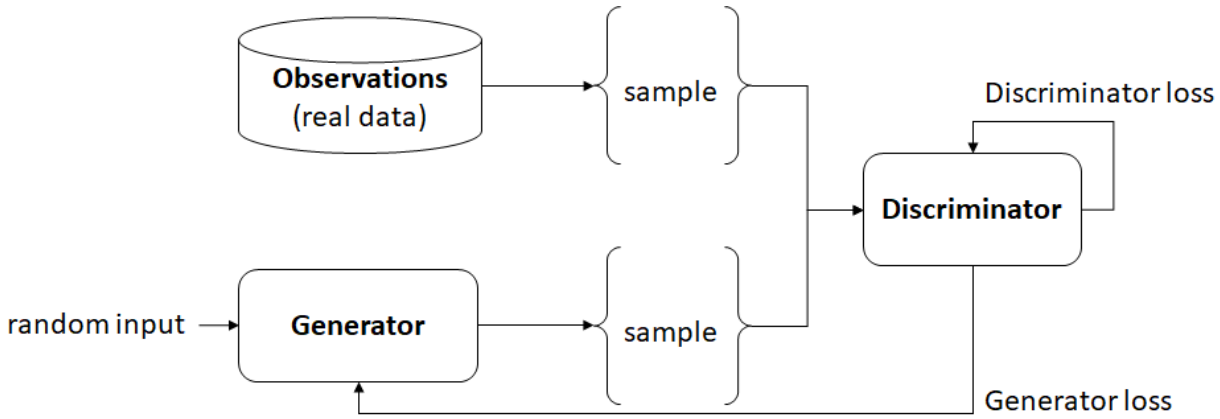


Nếu như Generative Adversarial Model là một tiến trình đi từ ngẫu nhiên đến đối tượng tạo sinh bằng cách sinh ra đống tượng chống lại (các) đối tượng thực; thì Diffusion Model lại bắt đầu bằng cách "vấy bẩn" (các) đối tượng bằng các nhiễu (noise), kiểu như tha giọt

mực vào ly nước lọc sẽ thấy các hạt mực phát tán ra "nhuộm bẩn" toàn bộ ly nước, và tiến trình tạo ra đối tượng mới là tiến trình tẩy nhiễu khỏi đối tượng bị nhiễu ban đầu.

Mô hình đối kháng tạo sinh

Mô hình đối kháng tạo sinh (generative adversarial model) gồm hai trình thành phần: trình tạo sinh (generator) và trình phân biệt (discirminator) hoạt động đan xen trong quá trình huấn luyện. Trong thực tiễn, các trình tạo xinh và phân biệt thường được cài đặt bằng mạng nơ-ron, bao hàm cả học sâu mạng sâu tầng, nên mô hình đối kháng được gọi chung là Mạng Đối kháng Tạo sinh (Generative Adversarial Networks), viết tắt là GAN như minh họa trong hình.



Một cách hình thức, Discriminator được huấn luyện để phân biệt một đối tượng đầu vào bất kỳ; trong khi đó, Generator được huấn luyện để tạo ra một đối tượng giả (fake object) sao cho Discriminator không phân biệt được thật giả. Đây là một nhiệm vụ khó. Generator học cách tạo ra dữ liệu hợp lý sẽ được sử dụng như các mẫu âm tính cho Discriminator học. Dismininator học cách phân biệt dữ liệu thật mà nó có, là mẫu dương tính, với các mẫu âm tính do Generator tạo ra. Quá trình đối kháng này diễn ra cho đến khi Discriminator không thể phân biệt được chính xác thật-giả.

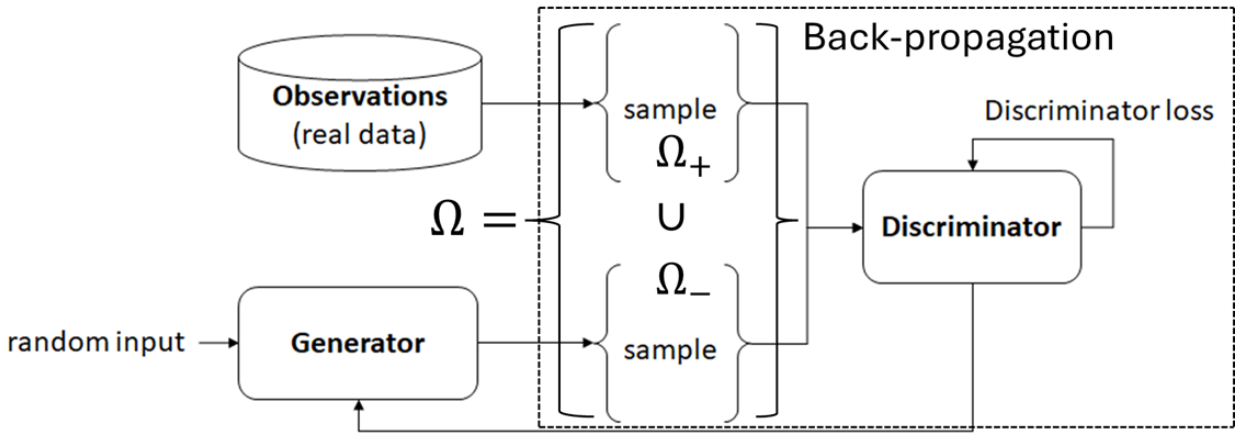
Discriminator là một mô hình học tất định kiểu Pred-AI, chỉ khác, thay vì dự đoán, nó tính xác suất phân biệt $p(fake|x)$ và $p(real|x)$ cho mỗi đầu vào x. Generator là mô hình học không tất định tìm cách học phân phối của dữ liệu để có thể gán xác suất $p(y, x)$ cho y là $fake$ hay $real$. Ta sẽ bắt đầu với Discrimonator cho dễ hiểu.

Discriminator

Discriminator chỉ đơn giản là một bộ phân loại $true/false$, mà cố gắng phân biệt dữ liệu thực (real data) mà nó biết với dữ liệu giả (fake data) do Generator tạo ra. Hồi quy logistic

(logistic regression), 3-layered neural network, hay bất kỳ mạng học sâu (deep learning) nào đều có thể sử dụng làm cấu trúc cho Discriminator.

Đóng băng Generator (xem hình vẽ), Discriminator được huấn luyện từ dữ liệu thực Ω_+ mà tất cả dữ liệu đều có nhãn là 1 ($true$) và tập dữ liệu âm tính, nhãn là 0 ($false$), Ω_- gồm các dữ liệu do Generator tạo ra. Để học tập mẫu $\Omega = \Omega_+ \cup \Omega_-$ này, kỹ thuật lan truyền ngược để cập nhật tham số theo đạo hàm hàm lỗi có thể được áp dụng.



Quá trình này gần như độc lập với Generator, ngoại trừ nó tính sẵn giá trị thưởng/phạt sẽ dùng cho Generator. Quy trình chung gồm các bước sau:

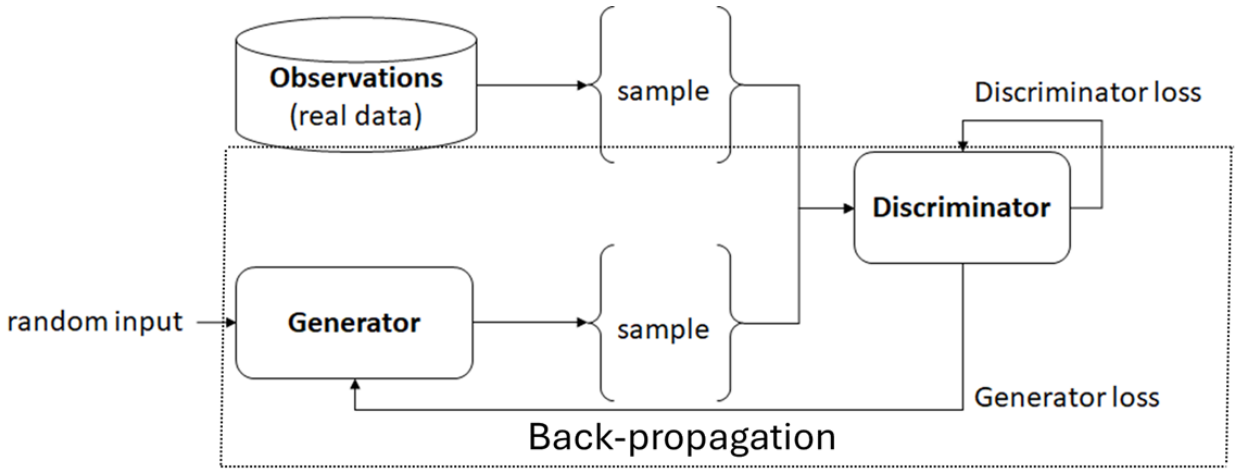
Step 1: Phân biệt dữ liệu từ Generator là thật hay giả.

Step 2: Tính giá trị đạo hàm hàm lỗi.

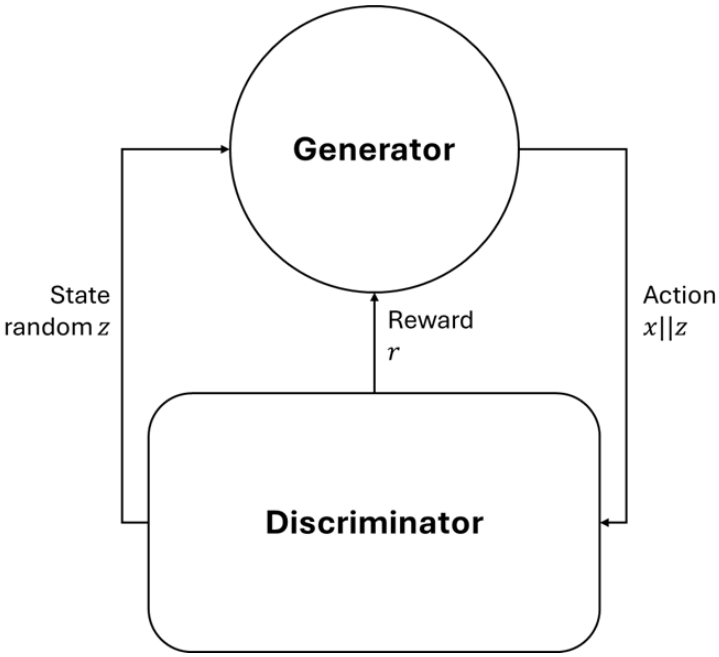
Step 3: Cập nhật trọng số của mạng theo trung bình giá trị đạo hàm hàm lỗi.

Generator

Bây giờ, đóng băng Discriminator không cho cập nhật trọng số của nó nữa mà chỉ sử dụng nó như "giám khảo" đánh giá xem đối tượng do Generator vừa tạo ra có còn giả nhiều không để quyết định phần thưởng cho hành động vừa tạo đối tượng đó.



Generator sẽ sử dụng kỹ thuật học tăng cường (reinforcement learning) để học các dữ liệu được phát sinh ngẫu nhiên bằng cách biên đổi giá trị ngẫu nhiên z thành một đối tượng dữ liệu $x||z$ ngẫu nhiên, sử dụng Discriminator để đáp ứng lại đối tượng đó bằng một phần thưởng (hay phạt) tùy thuộc độ thật-giả của đối tượng dữ liệu. Một reinforcement learning paradigm được thiết lập như sau:



. $s = z \sim P_\theta(z)$ là xác suất tạo ra z với tham số mạng Generator là θ.

. $y = D(x||z) = a$ là kết quả của Discriminator với dữ liệu sinh $x||z$.

. $r = R(x||z, y)$ sai số phân biệt.

Và có thể sử dụng thuật giải REINFORCE để cập nhật mạng Generator.

Thuật toán GAN

Như ta thấy, GAN gồm Generator và Discriminator, với các phương pháp huấn luyên khác nhau. Huấn luyện GAN là một tiến trình luân phiên giữa thuật toán học mạng tất định và thuật toán học mạng ngẫu nhiên:

GAN Algorithm:

Step 1: Đóng băng mạng Generator và huấn luyện Discriminator một số lần (epoch).

Step 2: Đóng băng mạng Discriminator và huấn luyện Generator một số lần (epoch).

(lặp lai Step 1 và Step 2 để huấn luyện GAN).

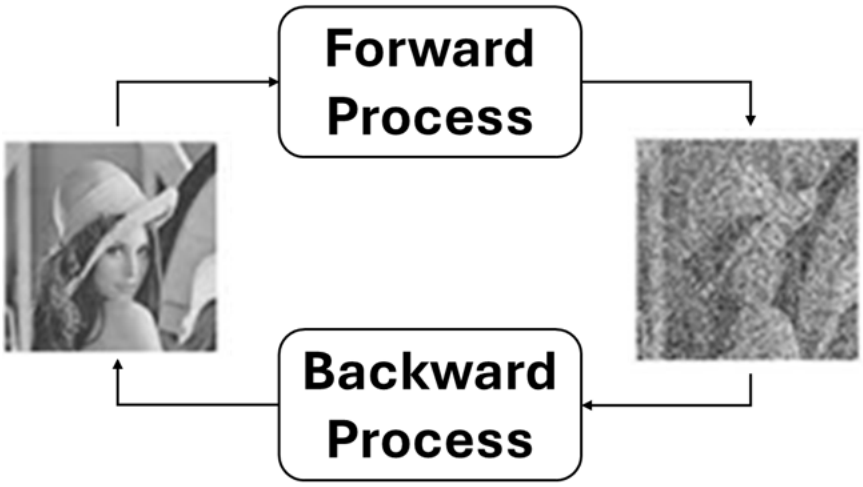
Câu hỏi đặt ra là lặp đến khi nào?

Trên nguyên tắc, hiệu suất của Generator được cải thiện đồng nghĩa hiệu suất phân biệt của Discriminator giảm. Như vậy, Generator được gọi là thánh công khi khả năng phân biệt của Discriminator là 50-50, nghĩa là lục phân biệt được, lúc không phân biệt được thật-giả. Theo đó, có thể thiết kế tiêu chí dừng cho thuật toán.

Mô hình phát tán

Khác với mô hình đối kháng tạo sinh (GAN), mô hình phát tán (Diffusion Models) làm việc bằng cách thêm nhiễu vào dữ liệu huấn luyện vào hóc cách đảo ngược tiến trình để "bóc" nhiễu ra. Sau đó, thao tác đảo ngược có thể được áp dụng cho dữ liệu ngẫu nhiên mới để tạo sinh ra kết quả mới.

Một cách tổng quát, Diffusion Model gồm 2 tiến trình (i) forward process và (ii) backward process như minh họa trong hình vẽ



(**i**) Forward Process nhận ảnh đầu vào và thực hiện tiến trình lặp thêm nhiễu vào ảnh cho đến khi thông tin ảnh gốc không còn (rõ ràng) nữa.

(**ii**) Backward Process nhận vào ảnh nhiễu và học cách tái tạo ảnh gốc bằng cách thực hiện tiến trình lặp khử nhiễu cho đến khi một ảnh mới (rõ ràng) được tạo ra.

Forward Process

Giả sử dữ liệu đầu vào là một véc-tơ thực d-chiều, $X_0 \in \mathbb{R}^d$, Forward Process (FP) liên tục thêm nhiễu để tạo ra dãy $X_1, X_2, \dots, X_T$ theo công thực đệ quy:

$$X_t = \sqrt{1-\beta_t} X_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t,$$

trong đó,

. $\boldsymbol{\varepsilon}_t$ là véc-tơ nhiễu Gauss ở bước lặp thứ t, và

. $0 < \beta_t < 1$ là tham số vô hướng. Dãy tham số $\beta_1, \beta_2, \dots, \beta_T$ được xác định trước và được gọi là lập lịch phương sai (variance schedule). Dãy $\beta_t, t = 1,2, \dots, T$ là dãy tăng, nhiểu càng về sau càng được thả vào nhiều hơn, nên FP là tiến trình tiến hóa.

Công thức đệp quy trên có thể được xác định như một tiến trình Markov liên kết với xác suất chuyển (transition density)

$$q_t = q(X_t|X_{t-1}) = \mathcal{N}(X_t;\ \sqrt{1-\beta_t} X_{t-1}, \beta_t \boldsymbol{I}),$$

với $\boldsymbol{I}$ là ma-trận đơn vị cấp d.

Đại lượng q_t chính là xác suất quan sát được X_t tại thời điểm t khi biết giá trị của X_{t-1} tại thời điểm $t-1$ ngay trước đó. Để xem X_t phụ thược 2 bước lặp trước,

$$X_t = \sqrt{1-\beta_t} X_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t = \sqrt{1-\beta_t}\,(\sqrt{1-\beta_{t-1}} X_{t-2} + \sqrt{\beta_{t-1}}\boldsymbol{\varepsilon}_{t-1}) + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t$$

Do $X \in \mathbb{R}^d$ là véc-tơ biến ngẫu nhiên rút ra từ phân phối chuẩn (Normal distribution or Gauss distribution),

$$\mathcal{N}(X; \boldsymbol{\mu}, \sigma\boldsymbol{I}) = \frac{1}{\sigma(2\pi)^{1/d}} \exp\left(-\frac{1}{2\sigma^2}(X-\boldsymbol{\mu})^T(X-\boldsymbol{\mu})\right),$$

với $\mu \in \mathbb{R}^d$ là véc-tơ trung bình (mean), và vô hướng (scalar) σ^2 là phương sai, và $\boldsymbol{I} \in \mathbb{R}^{d\times d}$ là ma-trận đơn vị.

Do biến ngẫu nhiên Gauss có tính cộng tính (additivity), nghĩa là 2 biến ngẫu nhiên Gauss độc lập X_1, X_2 từ hai phân phối $\mathcal{N}(X; \boldsymbol{\mu}_1, \sigma_1\boldsymbol{I})$ và $\mathcal{N}(X; \boldsymbol{\mu}_2, \sigma_2\boldsymbol{I})$, thì $X_1 + X_2$ có phân phối $\mathcal{N}(X; \boldsymbol{\mu}_1 + \boldsymbol{\mu}_2, (\sigma_1 + \sigma_2)\boldsymbol{I})$. Và nếu là phân phối Gauss chuẩn (standard Gauss), nghĩa là

$\boldsymbol{\mu} = \mathbf{0}, \sigma = 1$, thì nếu $\boldsymbol{y}, \boldsymbol{z}$ là 2 biến ngẫu nhiên Gauss chuẩn, a, b là 2 vô hướng, thì $a\boldsymbol{y} + b\boldsymbol{z}$ là biến ngẫu nhiên Gauss với $\boldsymbol{\mu} = \mathbf{0}$ và $\sigma = a^2 + b^2$. Vì thế $a\boldsymbol{y} + b\boldsymbol{z}$ có thể được lấy mẫu như $\sqrt{a^2 + b^2}\boldsymbol{\varepsilon}$, với $\boldsymbol{\varepsilon}$ là biến ngẫu nhiên Gauss chuẩn.

Trở lại với công thức ước lượng X_t phụ thuộc quan sát ở 2 bước trước

$X_t = \sqrt{1 - \beta_t}\,(\sqrt{1 - \beta_{t-1}}X_{t-2} + \sqrt{\beta_{t-1}}\boldsymbol{\varepsilon}_{t-1}) + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t$.

Bằng cách đặt

$\alpha_t = 1 - \beta_t$, ta viết lại

$X_t = \sqrt{\alpha_t\,\alpha_{t-1}}X_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\boldsymbol{\varepsilon}_{t-1} + \sqrt{1 - \alpha_t}\boldsymbol{\varepsilon}_t$

Với tính chất của Gauss chuẩn, tổng $\sqrt{\alpha_t(1 - \alpha_{t-1})}\boldsymbol{\varepsilon}_{t-1} + \sqrt{1 - \alpha_t}\boldsymbol{\varepsilon}_t$, và ký hiệu $\boldsymbol{\varepsilon}_{t,2}$ là phương sai Gauss chuẩn của biến ngẫu nhiên gộp, ta có thể viết

$X_t = \sqrt{\alpha_t\alpha_{t-1}}X_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\boldsymbol{\varepsilon}_{t,2}$.

Một cách đệ quy, với $\forall k$: $1 \le k \le t - 2$, thì

$X_t = \sqrt{\alpha_t\alpha_{t-1} \dots \alpha_{k+1}}X_k + \sqrt{1 - \alpha_t\alpha_{t-1} \dots \alpha_{k+1}}\boldsymbol{\varepsilon}_{t,k}$.

Đặt

$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i\,, \bar{\boldsymbol{\varepsilon}}_t = \varepsilon_{t,0}$.

Truy hồi về biến ngẫu nhiên đầu tiên X_0,

$X_t = \sqrt{\bar{\alpha}_t}X_0 + \sqrt{1 - \bar{\alpha}_t}\bar{\boldsymbol{\varepsilon}}_t$ hay $q(X_t|X_0) \sim \mathcal{N}(X_t;\ \sqrt{\bar{\alpha}_t}X_0, (1 - \bar{\alpha}_t)\boldsymbol{I})$.

Backward Process

Từ quan sát X_t và X_0, cần ước lượng X_{t-1}: $q(X_{t-1}|X_t, X_0)$.

Sử dụng công thức xác suất có điều kiện: $\Pr(A|B, C) = \frac{\Pr(B|A,C)\Pr(A|C)}{\Pr(B|C)}$, ta có

$q(X_{t-1}|X_t, X_0) = \frac{q(X_t|X_{t-1}, X_0)q(X_{t-1}|X_0)}{q(X_t, X_0)}$.

Xét $\frac{q(X_t|X_{t-1}, X_0)q(X_{t-1}|X_0)}{q(X_t, X_0)}$.

Do $X_0, X_1, \dots, X_t, \dots$ là xích Markov, ta có

. $q(X_t|X_{t-1}, X_0) = q(X_t|X_{t-1})$ hay $q(X_t|X_{t-1}, X_0) = \mathcal{N}(X_t; \sqrt{\alpha_t}X_{t-1}, (1 - \alpha_t)\boldsymbol{I})$,

. $q(X_{t-1}|X_0) = \mathcal{N}(X_t; \sqrt{\bar{\alpha}_{t-1}}X_0, (1-\bar{\alpha}_{t-1})\boldsymbol{I})$, và

. $q(X_t|X_0) = \mathcal{N}(X_t; \sqrt{\bar{\alpha}_t}X_0, (1-\bar{\alpha}_t)\boldsymbol{I})$,

Vậy

$$\frac{q(X_t|X_{t-1},X_0)q(X_{t-1}|X_0)}{q(X_t,X_0)} = \frac{\mathcal{N}(X_t;\sqrt{\alpha_t}X_{t-1},(1-\alpha_t)\boldsymbol{I})\mathcal{N}(X_t;\sqrt{\bar{\alpha}_{t-1}}X_0,(1-\bar{\alpha}_{t-1})\boldsymbol{I})}{\mathcal{N}(X_t;\sqrt{\bar{\alpha}_t}X_0,(1-\bar{\alpha}_t)\boldsymbol{I})}.$$

Từ định nghĩa $\mathcal{N}(X; \boldsymbol{\mu}, \sigma\boldsymbol{I}) = \frac{1}{\sigma(2\pi)^{1/d}} \exp\left(-\frac{1}{2\sigma^2}(X-\boldsymbol{\mu})^T(X-\boldsymbol{\mu})\right)$,

bỏ qua các số hạng hằng ta có thể viết các phân phối Gauss dưới dạng:

. $\mathcal{N}(X_t; \sqrt{\alpha_t}X_{t-1}, (1-\alpha_t)\boldsymbol{I}) \cong \exp\left(-\frac{1}{2}\frac{\left(X_t-\sqrt{\alpha_t}X_{t-1}\right)^T\left(X_t-\sqrt{\alpha_t}X_{t-1}\right)}{1-\alpha_t}\right)$,

. $\mathcal{N}(X_t; \sqrt{\bar{\alpha}_t}X_{t-1}, (1-\bar{\alpha}_t)\boldsymbol{I}) \cong \exp\left(-\frac{1}{2}\frac{\left(X_t-\sqrt{\bar{\alpha}_t}X_{t-1}\right)^T\left(X_t-\sqrt{\bar{\alpha}_t}X_{t-1}\right)}{1-\bar{\alpha}_t}\right)$,

. $\mathcal{N}(X_t; \sqrt{\bar{\alpha}_t}X_0, (1-\bar{\alpha}_t)\boldsymbol{I}) \cong \exp\left(-\frac{1}{2}\frac{\left(X_t-\sqrt{\bar{\alpha}_t}X_0\right)^T\left(X_t-\sqrt{\bar{\alpha}_t}X_0\right)}{1-\bar{\alpha}_t}\right)$.

Và $\frac{q(X_t|X_{t-1},X_0)q(X_{t-1}|X_0)}{q(X_t,X_0)}$ có dạng (*)

$$\exp\left(-\frac{1}{2}\frac{\left(X_t-\sqrt{\alpha_t}X_{t-1}\right)^T\left(X_t-\sqrt{\alpha_t}X_{t-1}\right)}{1-\alpha_t} - \frac{1}{2}\frac{\left(X_t-\sqrt{\bar{\alpha}_t}X_{t-1}\right)^T\left(X_t-\sqrt{\bar{\alpha}_t}X_{t-1}\right)}{1-\bar{\alpha}_t} + \frac{1}{2}\frac{\left(X_t-\sqrt{\bar{\alpha}_t}X_0\right)^T\left(X_t-\sqrt{\bar{\alpha}_t}X_0\right)}{1-\bar{\alpha}_t}\right).$$

Xét phân phối (**)

$\mathcal{N}(X_{t-1}; \mu_q(X_t, X_0), \sigma_q^2(t)\boldsymbol{I})$.

Trong đó

(i) $\sigma_q^2(t)$ được thiết lập bằng cách xét các hệ số của tích $X_{t-1}^T X_{t-1}$như trong dạng (*) trên. Các hệ thức này có dạng

$$\frac{1}{2}\cdot\frac{\alpha_t}{1-\alpha_t} + \frac{1}{2}\cdot\frac{1}{1-\bar{\alpha}_{t-1}} = \frac{1}{2}\cdot\frac{\alpha_t(1-\bar{\alpha}_t)+(1-\alpha_t)}{(1-\alpha_t)\cdot(1-\bar{\alpha}_t)} = \frac{1}{2}\cdot\frac{1-\alpha_t\bar{\alpha}_{t-1}}{(1-\alpha_t)\cdot(1-\bar{\alpha}_t)} = \frac{1}{2}\cdot\frac{1-\bar{\alpha}_t}{(1-\alpha_t)\cdot(1-\bar{\alpha}_t)},$$

với $\alpha_t\bar{\alpha}_{t-1} = \bar{\alpha}_t$. Vì thế

$\sigma_q^2(t) = \frac{(1-\alpha_t)\cdot(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t}$,

xác định với $\beta_t = 1-\alpha_t, t = 1,2,\dots,T$ được lập lịch trước.

(ii) và $\mu_q(X_t, X_0)$ được tính như sau.

Đặt V là véc-tơ thỏa $\frac{\mu_q(X_t,X_0)}{\sigma_q^2(t)} = V$, ta thấy

$\frac{\mu_q(X_t,X_0)}{\sigma_q^2(t)} = V = \frac{\sqrt{\alpha_t}X_t}{1-\alpha_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}X_0}{1-\bar{\alpha}_{t-1}}$.

Với $\sigma_q^2(t) = \frac{(1-\alpha_t)\cdot(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t}$ như đã thiết lập, suy ra

$\mu_q(X_t, X_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})X_t + \sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)X_0}{1-\bar{\alpha}_t}$.

Với $X_t = \sqrt{\bar{\alpha}_t}X_0 + \sqrt{1-\bar{\alpha}_t}\bar{\boldsymbol{\varepsilon}}_t$, ta có thể tính X_0 theo

$X_0 = \frac{X_t - \sqrt{1-\bar{\alpha}_t}\bar{\varepsilon}_t}{\sqrt{\bar{\alpha}_t}}$.

Suy ra

$\mu_q(X_t, X_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}X_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}}X_t - \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)\sqrt{1-\bar{\alpha}_{t-1}}}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}}\bar{\boldsymbol{\varepsilon}}_t$.

Do $\bar{\alpha}_{t-1}/\bar{\alpha}_t = 1/\alpha_t$ và $\alpha_t\bar{\alpha}_{t-1} = \bar{\alpha}_t$, tổng thừa số của 2 số hạng

$\frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} = \frac{\sqrt{\bar{\alpha}_t}\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1}) + \sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}}$

$= \frac{1}{\sqrt{\bar{\alpha}_t}(1-\bar{\alpha}_t)}(\alpha_t(1-\bar{\alpha}_{t-1}) + 1 - \alpha_t) = \frac{1}{\sqrt{\alpha_t}}$,

và thừa số của số hạng cuối

$\frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)\sqrt{1-\bar{\alpha}_{t-1}}}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} = -\frac{1-\alpha_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}$.

Cuối cùng, công thức ước lượng cho trung bình (mean)

$\mu_q(X_t, X_0) = \frac{1}{\sqrt{\alpha}}\left(X_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\bar{\boldsymbol{\varepsilon}}_t\right)$.

Trong biểu thức trên, $\bar{\boldsymbol{\varepsilon}}_t$ là nhiễu tích lũy trong quá trình biến đổi từ X_0 đến X_t, phải được dự đoán. Một hàm tham số hóa, có thể sử dụng một mạng nơ-ron, ký hiệu f_θ hay đơn giản chỉ cần θ, được xây dựng từ tập mẫu huấn luyện cho nhiệm vụ này. Quá trình huấn luyện là quá trình lặp. Tại mỗi bước lặp, lấy một mẫu trong tập huấn luyện, rút một véc-tơ nhiễu Gauss $\bar{\boldsymbol{\varepsilon}}_t$ và tạo mẫu nhiễu $X_t = \sqrt{\bar{\alpha}_t}X_0 + \sqrt{1-\bar{\alpha}_t}\bar{\boldsymbol{\varepsilon}}_t$, với $\bar{\alpha}_t = \Pi_{i=1}^t \alpha_i = \Pi_{i=1}^t(1-\beta_i)$,

với $\beta_1, \beta_2, \ldots$ được lập lịch trước. Nhiệm vụ của mạng là học để tạo ra $\boldsymbol{\varepsilon_\theta}$ gần với $\bar{\boldsymbol{\varepsilon}}_t$ nhất có thể.

Nói theo ngôn ngữ Reinforcement Learning, sử dụng một hàm được học θ theo $\mu_q(X_t, X_0) = \frac{1}{\sqrt{\alpha}}\left(X_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\bar{\boldsymbol{\varepsilon}}_t\right)$ để dự đoán nhiễu $\boldsymbol{\varepsilon}_\theta$, và với phương sai $\sigma_q^2(t) = \frac{(1-\alpha_t)\cdot(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t}$, mẫu được khử nhiễu theo công thức đệ quy

$$X_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(X_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\varepsilon}_\theta\right) + \sigma_q(t)\boldsymbol{z},$$

với $\boldsymbol{z}$ là véc-tơ Gauss chuẩn.

Thuật giải

Chúng tôi tóm tắt các tiến trình huấn luyện trong Forward Process và lấy mẫu trong Backward Process.

FP Algorithm:

Step 1: $X_0 \leftarrow \Omega$. Chọn một mẫu trong tập học.

Step 2: $t \leftarrow [T]$. Lấy ngẫu nhiên 1 giá trị trong tập $\{1, 2, \ldots, T\}$

Step 3: $\varepsilon \leftarrow \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Rút một véc-tơ nhiễu Gauss chuẩn.

Step 4: $\theta \leftarrow GD(\nabla_\theta)$. Cập nhật dùng đạo hàm trên $\left\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta\left(\sqrt{\bar{\alpha}_t}X_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\varepsilon}, t\right)\right\|_2^2$

(lặp các bước trên cho đến khi hội tụ).

BP Algorithm:

Step 1: $X_T \leftarrow \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Lấy mẫu Gauss chuẩn.

Step 2: $\forall t = T, T-1, \ldots, 1$:

Step 2.1: $\boldsymbol{z} \leftarrow \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

Step 2.2: $X_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(X_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\varepsilon}_\theta\right) + \sigma_q(t)\boldsymbol{z}$.

Step 3: Return X_0.